AADK 5 - Task 2 – Design a Kotlin Class & Data Model for an App Feature

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – **Session 5**

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1️⃣ Chosen Feature: Task Tracker

Description:
The Task Tracker feature allows users to create, manage, and mark daily tasks as completed. It is designed for students or professionals who want to stay organized and track productivity. This feature matters because it improves task visibility and encourages consistent goal completion.

2️⃣ Model the Data

✅ Data Class

```
data class Task(
    val id: Int,
    val title: String,
    val description: String,
    var isCompleted: Boolean = false
)
```

✅ Interface (Behavior Definition)

```
interface TaskManager {
    fun addTask(task: Task)
    fun removeTask(taskId: Int)
    fun toggleTaskCompletion(taskId: Int)
}
```

✅ **Implementation Class**

```
class TaskRepository : TaskManager {

    private val taskList = mutableListOf<Task>()

    override fun addTask(task: Task) {
        taskList.add(task)
    }

    override fun removeTask(taskId: Int) {
        taskList.removeAll { it.id == taskId }
    }

    override fun toggleTaskCompletion(taskId: Int) {
        taskList.find { it.id == taskId }?.let {
            it.isCompleted = !it.isCompleted
        }
    }

    fun getAllTasks(): List<Task> = taskList
}
```

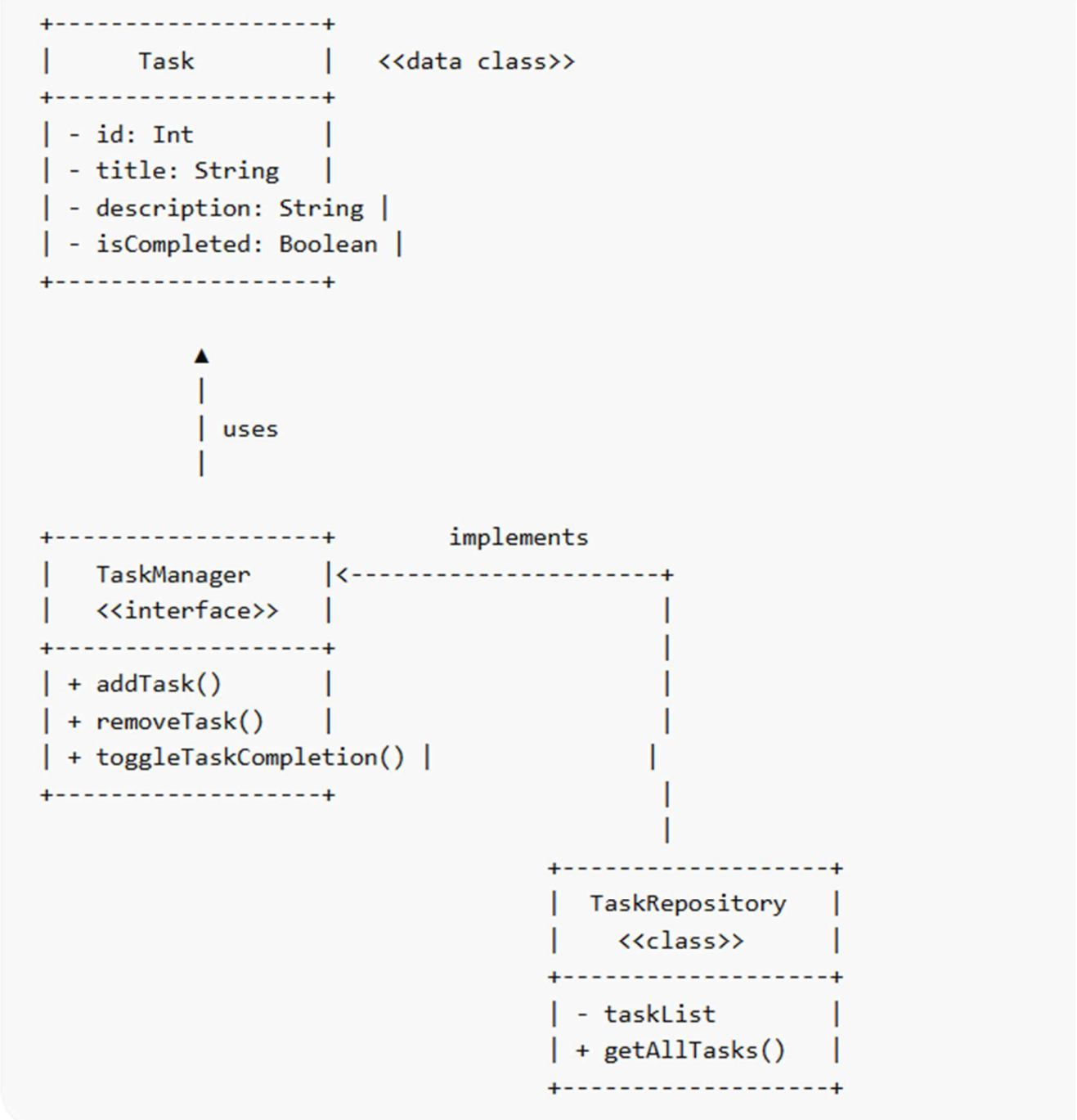
✅ **Why data class vs class vs interface?**

- **data class (Task)** is used because it primarily holds data. Kotlin automatically provides equals(), copy(), toString(), and hashCode(), which is ideal for state comparison in Compose.
- **interface (TaskManager)** defines behavior without implementation. This promotes abstraction and allows flexibility (e.g., later replacing repository with Room DB).

- **class (TaskRepository)** provides actual logic implementation and manages state.
- Using separation of concerns keeps business logic independent from UI.
- This structure improves scalability and testability.

---

3 UML / Sketch Diagram (Text Representation)



Kotlin Features Used:

- Primary constructor in Task
- Default parameter (isCompleted = false)
- Mutable property (var isCompleted)
- Interface implementation
- Collection filtering using lambda expressions

4️⃣ Mapping to Compose UI

In Jetpack Compose, the list of tasks would be displayed using LazyColumn. Each Task item would be shown inside a Card with a Checkbox. When toggleTaskCompletion() updates isCompleted, the state change would trigger recomposition if the list is backed by mutableStateListOf or State<List<Task>>. This ensures the UI automatically reflects task completion changes without manual refresh logic.

5️⃣ Reflection

Modeling behavior was more challenging than modeling data because it required thinking about responsibilities and separation of concerns. Instead of just storing properties, I had to design how tasks are added, removed, and updated safely. These object-oriented designs improve maintainability by keeping UI separate from business logic, making the app scalable and easier to test in the future.
